RL_CONCEPT

2026-03-28

Conceptual Process Description of the Final RL Asset Allocation Code

1. Problem setup

We study a finite-horizon discrete-time asset allocation problem with:

- one cash asset,
- n risky assets, where $n \in \{3, 4\}$,
- horizon $T \in \{1, \ldots, 9\}$.

Initial wealth is normalized to

$$W_0 = 1.$$

The portfolio is represented by weights

$$w_t = \left(w_t^{(0)}, w_t^{(1)}, \ldots, w_t^{(n)}\right),$$

where:

- $w_t^{(0)}$ is the fraction invested in cash,
- $w_t^{(k)}$ for $k = 1, \ldots, n$ is the fraction invested in risky asset k,
- all weights are nonnegative and sum to 1.

In the final code, the initial portfolio is chosen to be equal weight across all assets, including cash:

- if $n = 3$, then total assets $= 4$, so
  $w_0 = (0.25, 0.25, 0.25, 0.25)$
- if $n = 4$, then total assets $= 5$, so
  $w_0 = (0.2, 0.2, 0.2, 0.2, 0.2)$

The goal is to learn a policy that maximizes expected terminal utility under absolute risk aversion.

2. Financial assumptions

2.1 Risky asset returns

For each risky asset k, the one-period return is assumed to follow a normal distribution:

$$R_{t+1}^{(k)} \sim \mathcal{N}(a_k, s_k),$$

where:

- a_k is the mean return,
- s_k is the variance.

In the code, NumPy samples from a normal distribution using standard deviation, so the variance is converted by

$$\sigma_k = \sqrt{s_k}.$$

The code uses fixed parameter sets for the two values of n.

For $n = 3$:

$$a = (0.08, 0.10, 0.12)$$

$$s = (0.04, 0.09, 0.16)$$

For $n = 4$:

$$a = (0.08, 0.10, 0.12, 0.09)$$

$$s = (0.04, 0.09, 0.16, 0.05)$$

2.2 Cash return

Cash grows deterministically at rate r, with

$$r = 0.02.$$

2.3 No short-selling

Portfolio weights must remain nonnegative:

$$w_t^{(k)} \geq 0 \quad \text{for all } k.$$

2.4 Limited rebalancing

At each period, at most 10% of the portfolio can be adjusted. If the action is represented by a weight change vector

$$u_t = \left(u_t^{(0)}, u_t^{(1)}, \ldots, u_t^{(n)}\right),$$

then the code enforces the turnover constraint

$$\frac{1}{2}\sum_{k=0}^{n}\left|u_t^{(k)}\right| \leq 0.10.$$

The factor 1/2 is used because moving wealth from one asset to another counts once as selling and once as buying.

3. MDP formulation

The problem is modeled as a finite-horizon Markov Decision Process.

3.1 State

The state at time t is

$$s_t = (W_t, w_t),$$

where:

- W_t is current wealth,
- w_t is the current portfolio-weight vector.

This state is Markov because the next state depends only on:

- the current wealth,
- the current portfolio,
- the current action,
- the newly sampled returns.

3.2 Action

The action is a portfolio reallocation vector

$$u_t = \left(u_t^{(0)}, u_t^{(1)}, \ldots, u_t^{(n)}\right).$$

The post-trade portfolio is

$$\tilde{w}_t = w_t + u_t.$$

This must satisfy:

$$\sum_{k=0}^{n} u_t^{(k)} = 0,$$

$$\tilde{w}_t^{(k)} \geq 0 \quad \text{for all } k,$$

and

$$\frac{1}{2}\sum_{k=0}^{n} \left| u_t^{(k)} \right| \leq 0.10.$$

3.3 Transition dynamics

After rebalancing, returns are realized over one period.

Wealth evolves as

$$W_{t+1} = W_t \left[\tilde{w}_t^{(0)}(1+r) + \sum_{k=1}^{n} \tilde{w}_t^{(k)}(1+R_{t+1}^{(k)}) \right].$$

Then the next-period weights are obtained by normalizing post-return asset values.

For cash:

$$w_{t+1}^{(0)} = \frac{\tilde{w}_t^{(0)}(1+r)}{\tilde{w}_t^{(0)}(1+r) + \sum_{j=1}^{n} \tilde{w}_t^{(j)}(1+R_{t+1}^{(j)})}.$$

For risky asset k:

$$w_{t+1}^{(k)} = \frac{\tilde{w}_t^{(k)}(1+R_{t+1}^{(k)})}{\tilde{w}_t^{(0)}(1+r) + \sum_{j=1}^{n} \tilde{w}_t^{(j)}(1+R_{t+1}^{(j)})}.$$

3.4 Reward

There are no intermediate rewards. The only reward is at terminal time T, where the investor receives CARA utility of terminal wealth.

The code uses

$$U(W) = \frac{1-\exp(-\alpha W)}{\alpha},$$

with

$$\alpha = 2.$$

This utility is positive for positive wealth, but it is affine-equivalent to the more familiar CARA form $-\exp(-\alpha W)$, so it produces the same optimal policy.

The optimization objective is

$$\max_{\pi} \mathbb{E}_{\pi}[U(W_T)].$$

4. Why discretization is needed

The true MDP is continuous because:

- wealth is continuous,
- portfolio weights are continuous,
- actions are continuous in principle.

Tabular Q-learning requires a finite state and action space, so the code discretizes both.

5. Discretization used in the code

5.1 Weight grid

The code sets `WEIGHT_STEP = 0.05`.

This means all portfolio weights are represented on the grid

$$0,\ 0.05,\ 0.10,\ \ldots,\ 1.00,$$

subject to summing to 1.

Thus, the RL algorithm only stores values for portfolio weights on this finite simplex grid.

5.2 Wealth grid

Wealth is discretized on

$$0.50,\ 0.55,\ 0.60,\ \ldots,\ 2.00.$$

So if true wealth is not exactly one of these values, it is mapped to the nearest one.

5.3 Action grid

Actions are discretized as:

- do nothing,
- move 5% from one asset to another,
- move 10% from one asset to another.

This is consistent with the 10% turnover limit and creates a finite action set.

6. Nearest-grid approximation

Although the environment evolves in continuous values, the Q-table only stores values for discrete states.

So the code maps each true state (W_t, w_t) to the nearest grid state.

6.1 Wealth projection

If the true wealth is, for example,

$$W_t = 1.13,$$

and the grid step is 0.05, then the nearest wealth grid point is 1.15.

6.2 Weight projection

If the true weight vector is not exactly on the grid, the code selects the nearest admissible weight vector g that minimizes squared Euclidean distance:

$$\|w_t - g\|^2 = \sum_{i=0}^{n} \left(w_t^{(i)} - g^{(i)} \right)^2 .$$

This means the continuous state is approximated by the closest discrete state before looking up or updating the Q-table.

7. Structure of the code

The final code is organized into the following sections.

7.1 Global settings

This section defines:

- market parameters,
- discretization settings,
- learning parameters,
- convergence logging settings,
- evaluation settings,
- whether the optional DP comparison is run.

7.2 Basic functions

This section includes:

- the utility function,
- equal-weight portfolio construction,
- input validation.

7.3 Grid construction

This section generates:

- all admissible weight-grid states,
- all admissible actions.

7.4 State and action helper functions

These functions:

- discretize continuous wealth and weights,
- determine feasible actions,
- validate turnover and weights.

7.5 Environment transition

This section simulates one trading period:

1. apply the chosen rebalancing action,
2. sample risky returns,
3. update wealth,
4. normalize to obtain new portfolio weights.

7.6 Q-learning

This section performs finite-horizon tabular Q-learning with convergence tracking.

7.7 Benchmark policies

This section defines baseline policies for comparison:

- do nothing,
- random feasible action,
- greedy mean-return allocation,
- learned greedy policy.

7.8 Tests

The code includes:

- feasibility test,
- benchmark comparison,
- Bellman residual test,
- optional DP comparison.

7.9 Grid experiment loop

The code loops over all required assignment cases:

- $n = 3, 4$,
- $T = 1, \ldots, 9$,

and records outputs for every case.

8. Q-learning method

8.1 Time-dependent Q-table

Because the problem is finite horizon, the code uses a separate Q-table for each time step:

$$Q_t(s, a).$$

In code, this is stored as a 4-dimensional array indexed by:

- time t,
- wealth index,
- weight-state index,
- action index.

8.2 Episode initialization

Every episode starts from the same initial state:

$$W_0 = 1, \qquad w_0 = p.$$

Thus, changing the initial portfolio changes the training problem.

8.3 Action selection

The code uses epsilon-greedy exploration:

- with probability ε, choose a random feasible action,
- otherwise choose the action with the highest current Q-value.

The exploration parameter decreases over time according to

$$\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon \cdot \text{decay}).$$

8.4 Update rule

At each step, the code computes a target.

If $t = T - 1$, then the target is terminal utility:

$$\text{target} = U(W_T).$$

Otherwise, the target is the best next-step Q-value:

$$\text{target} = \max_{a'} Q_{t+1}(s_{t+1}, a').$$

The Q-value is then updated using

$$Q_t(s_t, a_t) \leftarrow (1 - \eta) Q_t(s_t, a_t) + \eta \cdot \text{target},$$

where η is the learning rate.

8.5 Multiple training runs

Because Q-learning is stochastic, the code trains multiple times with different random seeds. The best run is selected based on evaluation utility.

This improves robustness.

9. Convergence measurement in the updated code

The updated code now tracks the speed of convergence during training.

9.1 Motivation

Convergence speed describes how fast the learning algorithm stabilizes. In this code, convergence means that:

- Q-values stop changing much,
- temporal-difference errors become small,
- the policy becomes stable.

9.2 Fast convergence metrics used

To measure convergence with minimal extra runtime, the code records:

(a) Average Q-update size At each update, define

$$Q_{\text{new}} - Q_{\text{old}}.$$

The code records the magnitude

$$|Q_{\text{new}} - Q_{\text{old}}|\,.$$

A smaller value means the Q-table is changing less, suggesting convergence.

(b) Average TD error The temporal-difference error is

$$\delta = \text{target} - Q_{\text{old}}.$$

The code records

$$|\delta|.$$

A smaller average TD error means the learned Q-values are closer to their Bellman targets.

9.3 Why this method is fast

These quantities are already computed inside the Q-learning loop, so recording them adds almost no extra computational cost.

This is much faster than repeatedly evaluating the full policy during training.

9.4 Interpretation

If the algorithm is converging, then over time:

- average Q-update size should decrease,
- average TD error should decrease.

The final results table reports:

- `final_avg_q_update`,
- `final_avg_td_error`.

These summarize the convergence state near the end of training.

10. Main implementation assumptions

The code makes the following assumptions:

1. Independent normal returns. Risky asset returns are sampled independently from Gaussian distributions.
2. No transaction cost. The code restricts trading volume but does not subtract an explicit transaction fee.
3. No short-selling. All weights must remain nonnegative.
4. Finite-horizon control. The policy depends on time t.
5. Discrete approximation. Continuous states and actions are approximated by a finite grid.
6. Terminal-wealth objective. Only terminal CARA utility matters.
7. Equal-weight initial portfolio. The starting portfolio is equally weighted across all assets.

11. Tests used in the code

11.1 Feasibility test

This test checks whether the learned policy always satisfies the portfolio constraints.

It verifies:

- turnover is within 10%,
- post-trade weights are valid,
- next-period weights remain valid,
- wealth remains positive.

This is a correctness test of the implementation.

11.2 Benchmark comparison

The learned policy is compared with three baselines:

- do nothing,
- random feasible policy,
- greedy mean-return policy.

This gives an empirical performance comparison.

The code reports whether the learned policy beats:

- do nothing,
- random policy.

11.3 Bellman residual test

This test evaluates how well the learned Q-values satisfy the Bellman recursion.

For sampled state-action pairs, it compares:

- the current Q-value,
- the Monte Carlo estimate of the Bellman target.

The code reports:

- average Bellman residual,
- maximum Bellman residual.

Smaller residuals indicate better consistency.

11.4 Optional DP comparison

The code includes an optional dynamic-programming reference comparison.

This computes a model-based approximate optimal solution on the same discretized MDP and compares it with Q-learning.

Because this is computationally expensive, it is turned off by default and only intended for small representative cases.

12. Why the DP method is expensive

The DP comparison becomes slow because it must iterate over:

- all time steps,
- many wealth grid points,
- many weight-grid points,
- all feasible actions,
- many Monte Carlo transition samples.

With weight step 0.05, the weight-grid size becomes large, especially for $n = 4$. Therefore the DP method is much slower than Q-learning and is only used optionally.

13. Why the utility values are positive

The code uses

$$U(W) = \frac{1 - \exp(-\alpha W)}{\alpha}.$$

For positive wealth and positive α, this is positive because

$$\exp(-\alpha W) < 1.$$

So positive average terminal utility is expected.

This does not indicate a mistake, because the function is affine-equivalent to the usual negative CARA utility and yields the same optimal strategy.

14. Why the code loops over all (n, T) combinations

The assignment requires the code to work for:

- $n < 5$,
- $T < 10$.

Therefore the final script loops over:

- $n = 3$ and $n = 4$,
- $T = 1, 2, \ldots, 9$.

For each case, the code:

1. sets the correct parameters,
2. constructs the grids,
3. trains Q-learning,
4. evaluates the learned policy,
5. runs tests,
6. records the results in a summary table.

This demonstrates that the method works for all required cases.

15. Meaning of the output

For each pair (n, T), the code reports:

- average terminal wealth,
- standard deviation of terminal wealth,
- average terminal utility,
- feasibility test result,
- benchmark comparison results,
- Bellman residual statistics,
- final average Q-update size,
- final average TD error,

- optional DP comparison results,
- runtime.

Thus, the output includes both:

- performance measures,
- correctness and convergence diagnostics.

16. Overall method summary

The complete method can be summarized as follows.

1. Formulate the portfolio problem as a finite-horizon MDP.
2. Use state = current wealth + current portfolio weights.
3. Use action = feasible 5% or 10% reallocation.
4. Simulate one-step transitions using Gaussian risky returns and deterministic cash growth.
5. Use terminal CARA utility as the objective.
6. Discretize the continuous state and action spaces.
7. Train a time-dependent tabular Q-learning policy.
8. Track convergence using average Q-update size and average TD error.
9. Evaluate the learned policy across all required (n, T) cases.
10. Test the policy with feasibility checks, benchmark comparison, Bellman residuals, and optional DP comparison.

17. Final conclusion

The final submitted code provides a reinforcement-learning solution to the finite-horizon multi-asset allocation problem with limited rebalancing and absolute risk aversion. It is designed to handle all required assignment cases:

$$n \in \{3, 4\}, \qquad T \in \{1, \ldots, 9\},$$

using:

- a consistent MDP formulation,
- discretized tabular Q-learning,
- convergence tracking,
- structured testing.

The final output reports both utility performance and convergence/test diagnostics for every case.